

C program

```
1. C Program to Implement a Queue using an Array
2. */
3. #include <stdio.h>
4.
5. #define MAX 50
6. int queue_array[MAX];
7. int rear = -1;
8. int front = -1;
9. main()
10. {
11.     int choice;
12.     while(1)
13.     {
14.         printf("1.Insert element to queue \n");
15.         printf("2.Delete element from queue \n");
16.         printf("3.Display all elements of queue \n");
17.         printf("4.Quit \n");
18.         printf("Enter your choice : ");
19.         scanf("%d",&choice);
20.         switch(choice)
21.         {
22.             case 1:
23.                 insert();
24.                 break;
25.             case 2:
26.                 delete();
27.                 break;
28.             case 3:
29.                 display();
30.                 break;
31.             case 4:
32.                 exit(1);
33.             default:
34.                 printf("Wrong choice \n");
35.         } /*End of switch*/
```

```

36.     }/*End of while*/
37.     }/*End of main()*/
38.     insert()
39.     {
40.         int add_item;
41.         if(rear == MAX -1)
42.             printf("Queue Overflow \n");
43.         else
44.         {
45.             if(front ==-1)
46.                 /*If queue is initially empty */
47.                 front =0;
48.             printf("Inset the element in queue : ");
49.             scanf("%d",&add_item);
50.             rear = rear +1;
51.             queue_array[rear]= add_item;
52.         }
53.     }/*End of insert()*/
54.
55.     delete()
56.     {
57.         if(front ==-1|| front > rear)
58.         {
59.             printf("Queue Underflow \n");
60.             return;
61.         }
62.         else
63.         {
64.             printf("Element deleted from queue is : %d\n",
queue_array[front]);
65.             front = front +1;
66.         }
67.     }/*End of delete() */
68.     display()
69.     {
70.         int i;
71.         if(front ==-1) || (front > rear)

```

```
72.     printf("Queue is empty \n");
73.     else
74.     {
75.         printf("Queue is : \n");
76.         for(i = front; i <= rear; i++)
77.             printf("%d ", queue_array[i]);
78.         printf("\n");
79.     }
80.     }/*End of display() */
```

In a theme park, the Roller-Coaster ride is started only when a good number of riders line up in the counter (say 20 members). When the ride proceeds with these 20 members, a new set of riders will line up in the counter. This keeps continuing. Implement the above scenario of lining up and processing using arrays with Queue ADT.

Low Level: Implement the aforementioned problem [6 Marks]

Middle Level: Also count the number of adults and children [2 Marks]

High Level: If a VIP family arrives, process them before processing others who are already waiting in the queue [2 marks]